

Prompts that Learn: Heuristic-Guided Prompt Engineering via Expectation Maximization in LLM Workflows

Mark Schutera

mark.schutera@mailbox.org

Abstract

We ask whether a *small* (3B-parameter) local LLM, used as the per-car driving policy with *all* knowledge in the prompt, can match crowd-sourced deep RL on MIT DeepTraffic. On a frame-exact re-implementation of the benchmark (11-car, official 500×2000 deterministic protocol; score = median fleet mph), a prompt-only `qwen2.5:3b` policy reaches a 24-run median of **76.5 mph** (point estimate; expected median ~ 76.4 , $P(>76.3)=0.85$), with a mean (76.13) matching the best hand-tuned heuristic’s 500-run mean (76.12) and exceeding the top real DQN submission (74.13) by ~ 2.4 mph. We then formalize the systematic *manual* tuning that achieved this as **prompt-EM**: a regression/EM loop that co-optimizes the *weighting* and the *ordering* of a small set of human-written driving heuristics, which an LLM synthesizes into one prompt, scored each round by the simulator’s median speed as a closed-loop feedback signal. Run at scale over 16 heuristics, prompt-EM recovers a strong prompt automatically (proxy median 75.9, well above random search), the residual gap to hand-tuning being prompt *wording*. We release the suite with full provenance and per-iteration logging so that heuristic importance and *sequence* importance can be analyzed.

1 Setup

DeepTraffic scores the median, over many deterministic episodes, of the average speed of a fleet of identical agents weaving through slower traffic. Our engine reproduces the original JavaScript bit-for-bit (18/18 fidelity tests pass; max per-frame deviation 0.0, single- and multi-agent). Reference points on this engine: do-nothing floor ~ 60 ; best real trained-DQN submission **74.13**; archived leaderboard #1 **75.66**; published all-time ceiling **76.3**. The repo’s prior-best heuristic sits *below* this ceiling (expected median 76.17); a new hand-coded convention sweep gives an expected median of **76.40** (5 SE above 76.3 over ten independent 500-run blocks), the first policy here to clear it. A second, more aggressive sweep finds *no* configuration above ~ 76.5 , so 76.3 sits near the engine’s practical optimum for *any* policy.

2 A small LLM at the ceiling

The policy receives a *lossy semantic rendering* of the same observation the DQN sees—the nearest car per lane within the ego’s lane and its two neighbours, as “open / fast-far / slow-close” plus an explicit rear status—and a short prose driving convention; it reasons one sentence and emits one of five actions. At temperature 0 it is a deterministic function of the prompt, so an exact-text cache is decision-identical to querying every step (verified: pure uncached runs reproduce cached scores exactly). The tuned prompt reaches a **24-run median of 76.50** (mean 76.13, matching the heuristic’s 500-run mean 76.12); a bootstrap places $P(\text{median} > 76.3) \approx 0.85$. Three findings drove this from a 72.3 naive baseline: (i) a 3B model follows a *prose* convention but fails numbered numeric rule-lists; (ii) agreement with the heuristic oracle does *not* track score—*blocked-state* decision quality does; (iii) small wording and *ordering* changes swing several mph, because the model anchors on whichever rule is stated first/most forcefully.

3 Prompt-EM

We systematize this. Let $h_{1:K}$ be atomic heuristics, each a one-sentence rule. A genome carries two simplex vectors: emphasis α (which rules, how strongly) and priority ρ (their order). Each iteration: (1) an LLM **synthesizes** one system prompt that states the heuristics in ρ -order, stressing high- α ones (a deterministic template is the fallback); (2) **evaluate** over 10 runs $\rightarrow$ median; (3) **E-step**: ridge-regress the objective on past α (and on ρ) to estimate each heuristic’s marginal contribution and its order-contribution; (4) **M-step**: $\alpha \leftarrow (1-\eta) \alpha_{\text{best}} + \eta r(\alpha)$, moving toward the positive-contribution *mixture* (not the single best rule), with learning rate η and annealed exploration. A Latin-hypercube cold start makes the otherwise-collinear regression identifiable (synthetic tests recover the true helpful/harmful rules in ~ 12 iterations). Single heuristics map to prose *1-to-1* (no LLM); only the weighted *blend* uses LLM synthesis. The state renderer is held fixed—we optimize the rules, not the encoding.

Prompt-EM in practice. A 3B model is a high-variance *synthesizer*: free-form LLM synthesis of the blended prompt collapses the policy into a passive “maintain” mode (40–74 mph) because it fails to reliably state the dominant “accelerate-by-default” rule. Baking the proven wording into the heuristics and using the deterministic template removes this variance (stable 74.6). The EM cold start then doubles as an *automatic ablation*: zeroing the accelerate rule’s

weight collapses the fleet to 45 mph, cleanly marking it the dominant heuristic. Over $K=16$ heuristics the exploit phase reaches a 1000-frame-proxy median of **75.9** (mean 75.0 vs. 72.9 for the random Latin-hypercube cold-start samples): at this budget the loop’s gain over random search is *reliability*—it concentrates in the good region—rather than a higher peak.

Positioning. Prompt-EM is black-box prompt optimization (cf. APE [1], OPRO [2], EvoPrompt [3], Promptbreeder [4], APO/ProTeGi [5], TextGrad [6], DSPy [7]) applied to an LLM-as-driving-policy (cf. DiLu [8], GPT-Driver [9], LanguageMPC [10]). Its distinguishing combination is an *interpretable* search space—a weight-and-order simplex over human heuristics, not free-form or soft-prompt text—updated by a *regression/EM* credit assignment that reports *which* rules and *which positions* the loop’s feedback favors; instruction-order credit is absent from prior black-box prompt optimizers, and the objective is closed-loop multi-agent control.

4 Limitations

The margin over 76.3 is necessarily thin (the engine ceiling is ~ 76.5) and our 24-run significance is suggestive (85%), not yet 95%. Credit assignment over K heuristics from few in-loop evaluations is sample-limited, and emphasis and order confound under joint search. The policy and the DQN share an observation *source* but not identical input features: the renderer is fixed and information-non-increasing—no privileged data reaches the policy—yet it exposes only a hand-thresholded slice of the 315-cell grid (nearest car per lane, ego ± 1 lanes), so part of the policy’s competence sits in this feature map rather than in the model. The automated loop reaches the mid-75s without any manual wordsmithing but does not yet match the hand-tuned 76.5: the residual gap is prompt *wording*, which neither the weight nor the order channel reaches. Closing it—a staged synthesizer *freedom* that explores wording around a frozen best structure—and isolating sequence importance with a dedicated order-only run are immediate future work; both are implemented as ready hooks. All runs ship with a full reproducibility manifest and per-iteration logs (weights, order, action distributions, contributions).

References

- [1] Zhou et al. Large Language Models Are Human-Level Prompt Engineers (APE). arXiv:2211.01910, 2022.
- [2] Yang et al. Large Language Models as Optimizers (OPRO). arXiv:2309.03409, 2023.
- [3] Guo et al. EvoPrompt. arXiv:2309.08532, 2023.
- [4] Fernando et al. Promptbreeder. arXiv:2309.16797, 2023.
- [5] Pryzant et al. Automatic Prompt Optimization (APO/ProTeGi). arXiv:2305.03495, 2023.
- [6] Yuksekgonul et al. TextGrad. arXiv:2406.07496, 2024.
- [7] Khattab et al. DSPy. arXiv:2310.03714, 2023.
- [8] Wen et al. DiLu. arXiv:2309.16292, 2023.
- [9] Mao et al. GPT-Driver. arXiv:2310.01415, 2023.
- [10] Sha et al. LanguageMPC. arXiv:2310.03026, 2023.